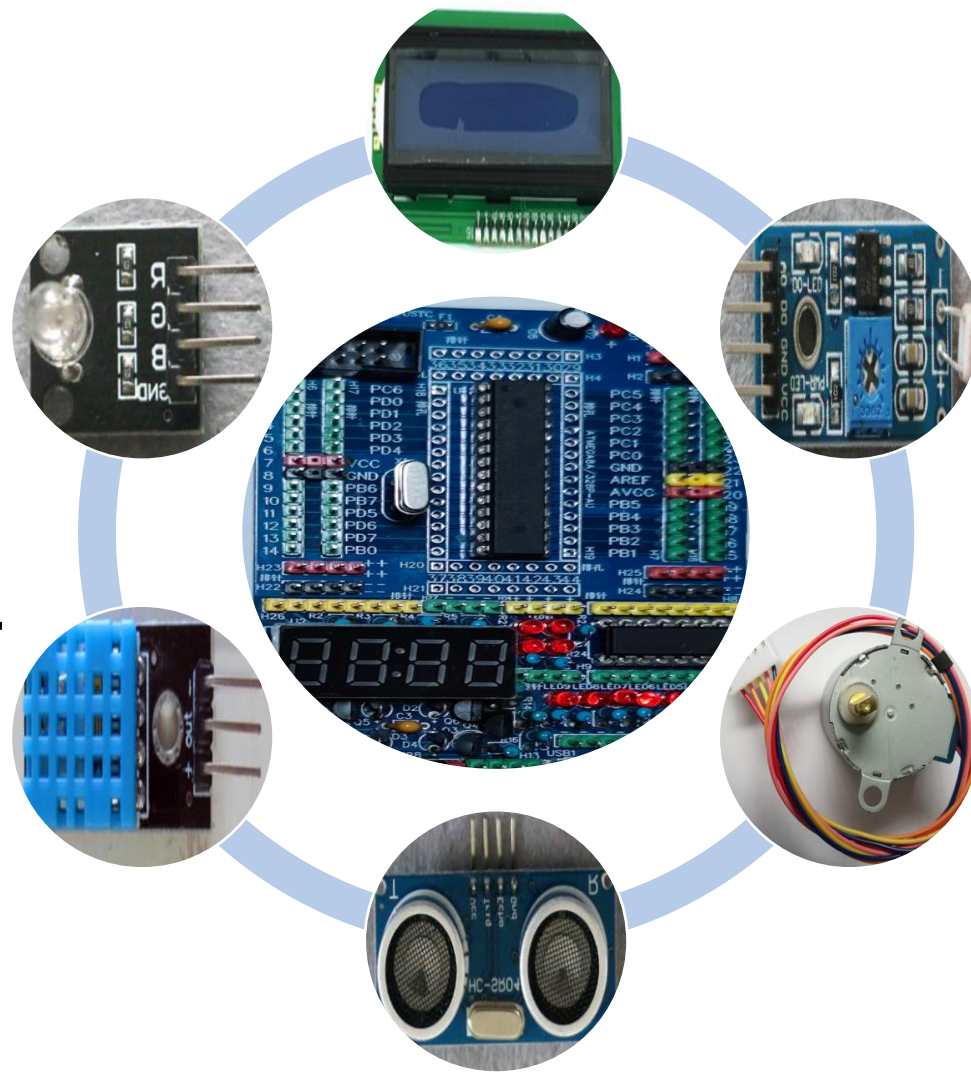


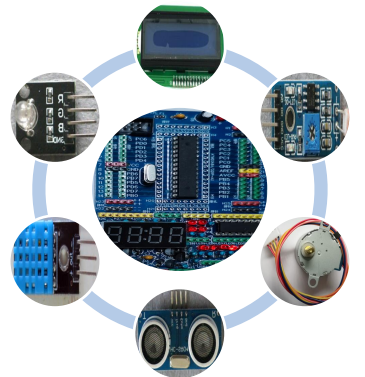
电子设计实践 基础

MCU简单任务调度、中
断与小键盘、LCD命令与
汉字、蜂鸣器发声等



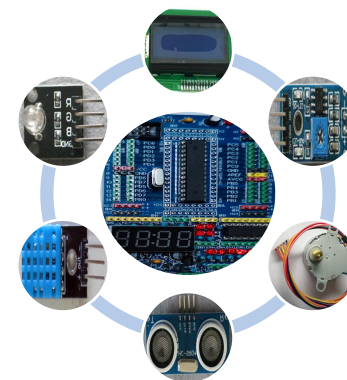
上节课内容回顾

- ATMEGA8A的加密位（保护芯片内的程序）
- ATMEGA8A的熔丝位（设置工作时钟等）
- ATMEGA8A的USART接口（通用串口）



本节课主要内容

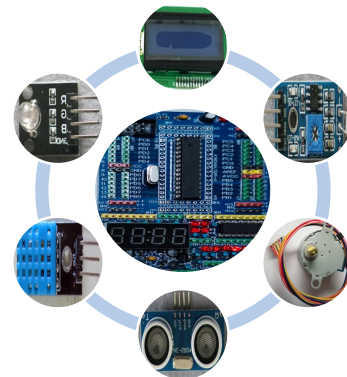
- MCU简单任务调度-MCU的“OS” (打破常规)
- 定时器中断与矩阵键盘
- LCD命令与汉字 (自定义字符)
- 蜂鸣器发声



MCU简单任务调度1：MCU的“OS”——打破常规

- MCU的“OS”？别逗了
- 避免MCU大量的长延时(空转)，提高MCU的利用率
- 任务调度：时间片、轮转、多个任务、调度（优先、抢占...）；MMU、...

示例



MCU简单任务调度2：示例（1）

```
#include <avr/io.h>
#include <avr/interrupt.h>
#include "twi_lcd.h"
#define MAX_TASKS 3
typedef struct {
    void(*ftask)(void); //指向函数的指针，即要执行的任务
    int ticks;           //执行此任务的时间间隔
}OSTASK;
unsigned char tp_cnt=0; //统计触摸开关的开关次数
void tp_task()          //统计触摸开关的开关次数
{
    static unsigned char tp1=0,tp2=0; //读取开关的状态,保留上一次的结果
    DDRB &= ~(1<<DDRB0);             //PB0管脚连接触摸开关的开关信号
    tp2 = tp1;                        //缓存
    tp1 = (PINB & (1<<PINB0));        //读取PB0管脚的状态
    if(tp2==0 && tp1==1)tp_cnt++;     //统计开关次数
}
```

MCU简单任务调度2：示例（2）

```
void Rgb_show()
{
    DDRC |= (1 << DDRC0) | (1 << DDRC1) | (1 << DDRC2); //为输出
    switch(tp_cnt) {                                     //根据触摸开关的开关次数点亮LED
        case 0: PORTC |= (1 << PORTC0); break;
        case 2: PORTC |= (1 << PORTC1); break;
        case 4: PORTC |= (1 << PORTC2); break;
        case 1: PORTC &= ~(1 << PORTC0); break;
        case 3: PORTC &= ~(1 << PORTC1); break;
        case 5: PORTC &= ~(1 << PORTC2); break;
        case 6: PORTC |= (1 << PORTC0); break;
        case 7: PORTC |= (1 << PORTC1); break;
        case 8: PORTC |= (1 << PORTC2); break;
        case 9: PORTC &= ~(1 << PORTC2); break;
        default: tp_cnt = 0; break;
    }
}
```

MCU简单任务调度2：示例（3）

```
void lcd_show()
{
    char ch[5]={ '0'},i;           //
    unsigned char t=tp_cnt;
    for(i=0;i<3;i++)
    {
        ch[2-i]=t%10+48;           //数字转换为字符
        t/=10;                     //去掉低位
    }
    cli();                          //全局中断关，不然会影响显示过程，定时器0中断服务也停止了
    LCD_Write_String(0,1,"times:");
    LCD_Write_String(0,8,ch);
    sei();                          //全局中断开
}

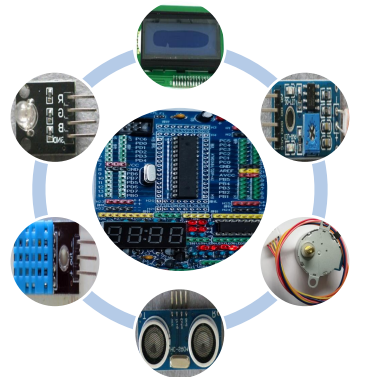
int ticks[MAX_TASKS]={3,7,19};     //对应于任务执行间隔（时间片），单位1ms
OSTASK_runtask[MAX_TASKS]= {{tp_task,3},{Rgb_show,7},{lcd_show,19}};
//任务与执行时间操作列表，方便循环处理
```

MCU简单任务调度2：示例（4）

```
int main(void)
{
    unsigned char i=0;
    TWI_Init();      LCD_Init();           //lcd1602初始化
    //定时器0的初始化：1ms的定时与中断，用做时间片基准，中断定时等
    TCNT0 = 131;      //1MHz的8分频，从131计数到255中断一次正好1ms
    TIMSK = (1<<TOIE0);           //定时器0的溢出中断开
    TCCR0 = (1<<CS01);           //CPU时钟的8分频，定时器0开始工作
    sei();                     //全局中断开
    while (1) {
        for(i=0;i<MAX_TASKS;i++) {           //调度
            if(runtask[i].ticks==0) {         //到达此任务的执行（时间片耗尽）
                runtask[i].ticks=ticks[i];    //恢复此任务的时间片
                runtask[i].ftask();           //执行相应的任务
            }
        }
        for(i=0;i<100;i++);                  /*延时*/
    } }
    //while和main结束
```


MCU简单任务调度2： 示例（5）

```
ISR(TIMERO0_OVF_vect) //1MHz的8分频，从131计数到255中断一次正好1ms
{
    TCNT0 = 131;           //溢出后TC0从初始值重新计数
    for(unsigned char i=0;i<MAX_TASKS;i++)
        runtask[i].ticks--; //时间片调整
}
```



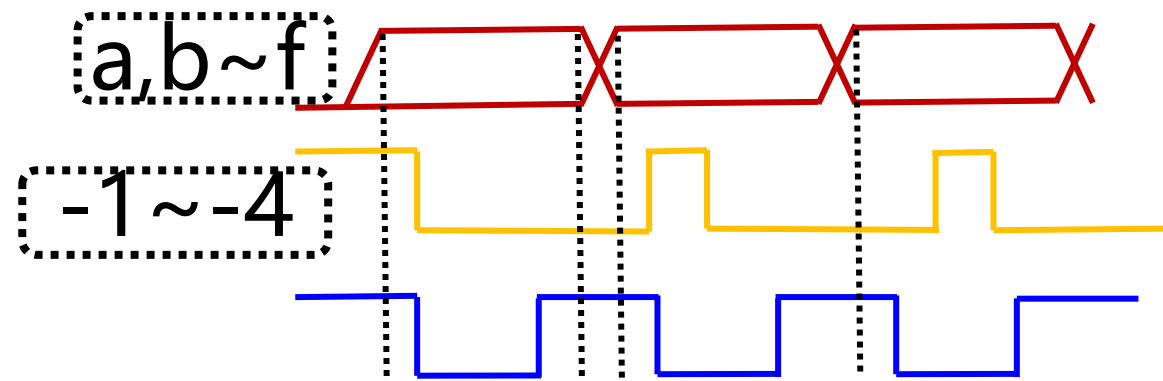
定时器中断与小键盘1：回顾（1）问题与处理

■七段数码管动态扫描显示会出现叠加现象

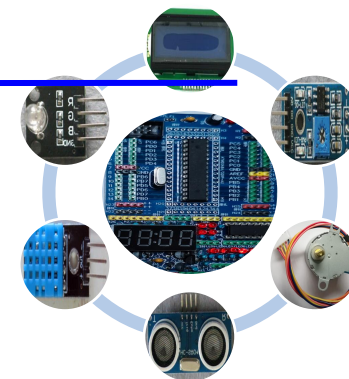
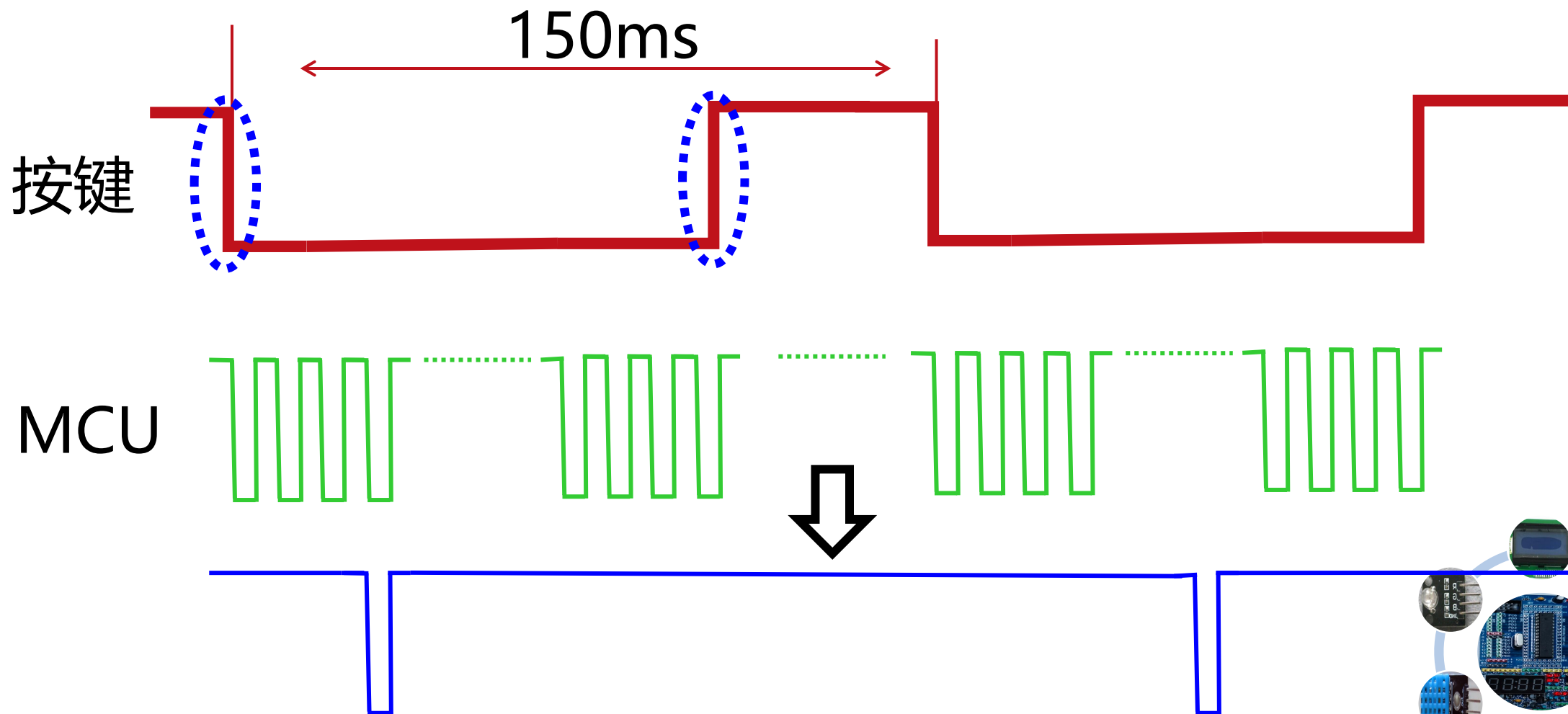
- 避免数码管选中时数据抖动

■按键阵列输入时会出现MCU处理时间与按键按下时间不匹配的情况，即显示重复输入现象

- 降低处理时间：统计、延时
- 改进处理方式：边沿、中断



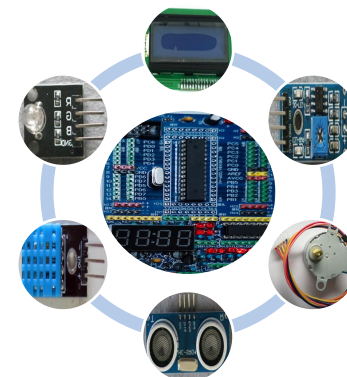
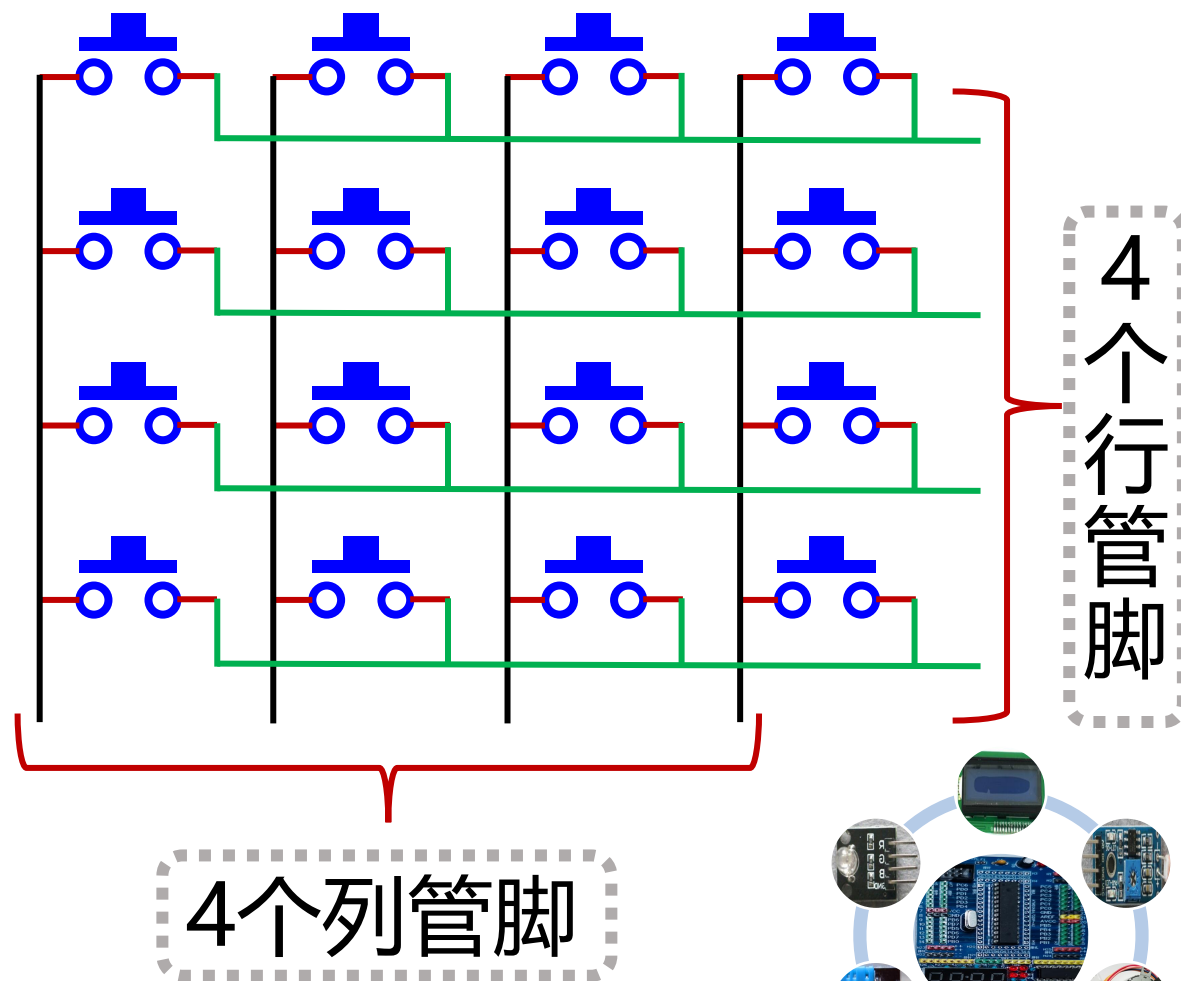
定时器中断与小键盘1：回顾（2）解决原理



定时器中断与小键盘1：回顾（3）-结构与管脚分布1

S_C3 1
S_C2 2
S_C1 3
S_C0 4
S_R0 5
S_R1 6
S_R2 7
S_R3 8

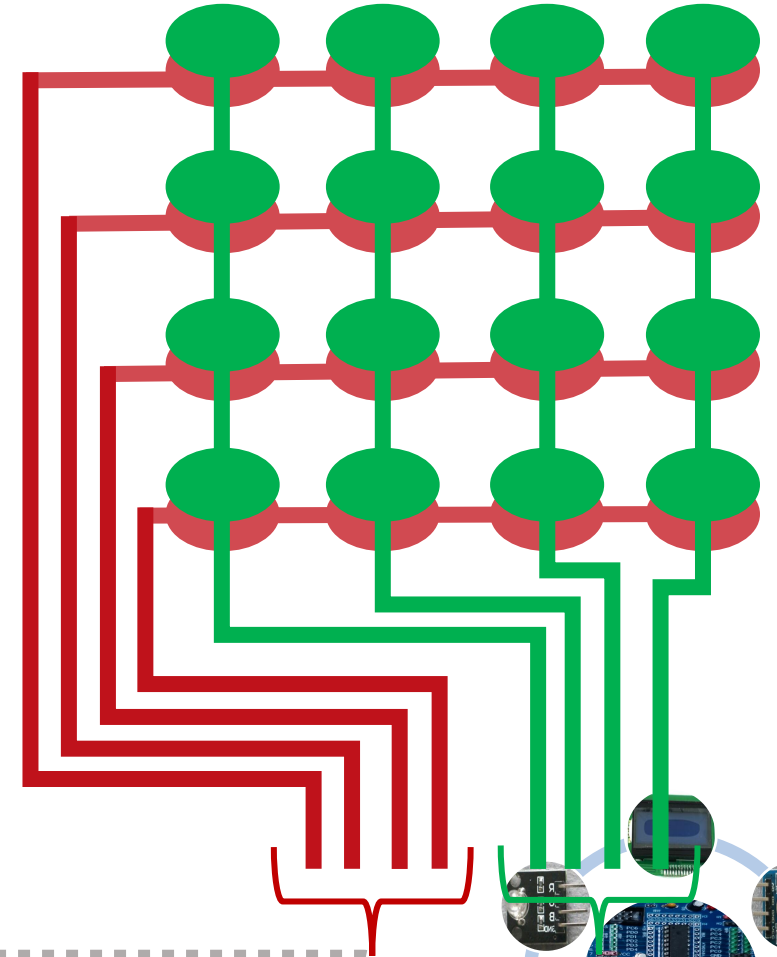
管脚
分布



定时器中断与小键盘1：回顾（3）-结构与管脚分布2

S_R0 1
S_R1 2
S_R2 3
S_R3 4
S_C0 5
S_C1 6
S_C2 7
S_C3 8

管脚
分布



4个行管脚

4个列管脚

定时器中断与小键盘1：回顾

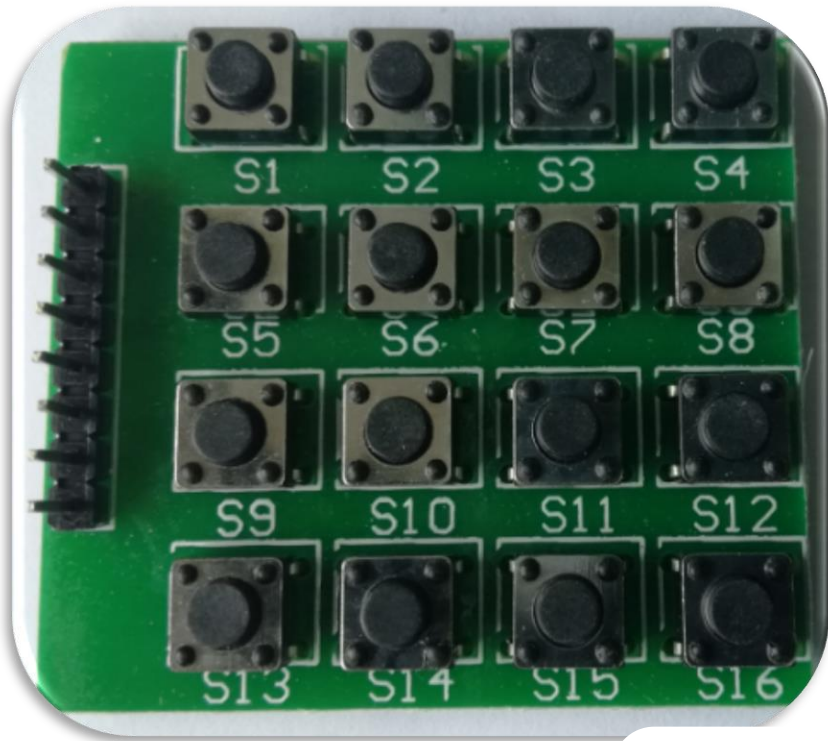
(4) - 中断向量表

- 不同的中断源
- 不同的中断入口地址
- 优先级
 - (中断号小的优先级高)
 - 优先级高的可以打断优先级低的服务过程

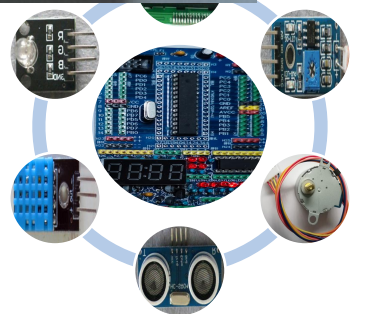
中断号	入口地址	中断源	中断说明
1	0x000	RESET	外部, 上电, 掉电或看门狗复位
2	0x001	INT0	外部中断请求0
3	0x002	INT1	外部中断请求1
4	0x003	TIMER2 COMP	定时器/计数器2 比较匹配
5	0x004	TIMER2 OVF	定时器/计数器2 溢出
6	0x005	TIMER1 CAPT	定时器/计数器1 捕获事件
7	0x006	TIMER1 COMPA	定时器/计数器1 比较匹配A
8	0x007	TIMER1 COMPB	定时器/计数器1 比较匹配A
9	0x008	TIMER1 OVF	定时器/计数器1 溢出
10	0x009	TIMER0 OVF	定时器/计数器0 溢出
11	0x00A	SPI,STC	串行传输结束
12	0x00B	USART,RXC	USART, Rx接收结束
13	0x00C	USART,UDRE	USART数据寄存器空
14	0x00D	USART,TXC	USART, Tx发送结束
15	0x00E	ADC	ADC转换结束
16	0x00F	EE_RDY	EEPROM准备好
17	0x010	ANA_COMP	模拟比较器
18	0x011	TWI	两线串行接口
19	0x012	SPM_RDY	保存程序存储器准备好

定时器中断与小键盘2：实例-硬件连接

中断方式将小键盘最近四次按下的键，按顺序编码并显示到LCD



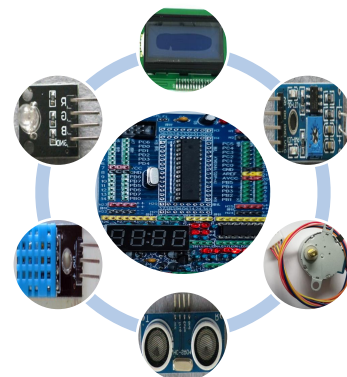
同时为了连线的方便，可改动接线顺序，编码顺序等



定时器中断与小键盘2：实例-编程（1）

。。。#define与#include等省略。。。

```
unsigned char hexStr[17]={'0','1','2','3','4','5','6','7','8','9','A','B','C','D','E','F','-'};
unsigned char key_no[5]={16,16,16,16,0}; //存放4位连续按下的编码,以' \0'结束
unsigned char getkey,keyno; //取按键阵列扫描输入,按下按键的编码
int main(void) { DDRD = (0xf0); //PD7~4为输出S_R0~3, PD3~0为输入S_C0~3
    PORTD = (0xff); //PD7~4输出高电平,PD3~0的内部上拉电阻启用
    TCCR0 = (1<<CS02)|(1<<CS00); //1024分频
    TCNT0 = 102; //从102开始计数, 1MHz/1024/154约为150ms中断一次
    TIMSK |= (1<<TOIE0); //允许TOV0中断
    TWI_Init(); LCD_Init();
    sei(); //全局中断开
    while (1) { LCD_Write_Char(0,1,hexStr[key_no[3]]);
        LCD_Write_NewChar(hexStr[key_no[2]]);
        LCD_Write_NewChar(hexStr[key_no[1]]);
        LCD_Write_NewChar(hexStr[key_no[0]]);
        _delay_ms(20); } }
```



定时器中断与小键盘2：实例-编程（2）

```
ISR(TIMERO0_OVF_vect)           //溢出中断TCNT0=255溢出到102
{ TCNT0 = 102;                   //从102开始计数, 1MHz/1024/154约为150ms中断一次
  keyno = 16;                    //默认无按键按下
  //1.扫描第1行
  PORTD = ~(1<<PORTD7);        //PORTD7为0,其它为1, 送给第1行
  _delay_us(2);
  getkey = (PIND & 0x0f);       //取按键阵列的列状态
  switch(getkey) {
    case 0x07:keyno = 0;break;  //1行1列编码为0/1 //S4
    case 0x0b:keyno = 1;break;  //1行2列编码为1/2 //S8
    case 0x0d:keyno = 2;break;  //1行3列编码为2/3 //S12
    case 0x0e:keyno = 3;break;  //1行4列编码为3/A //S16 }
  //2.扫描第2行
  PORTD = ~(1<<PORTD6);        //PORTD6为0,其它为1, 送给第2行
  _delay_us(2);
  getkey = (PIND & 0x0f);       //取按键阵列的列状态
```

• • • • •

定时器中断与小键盘2：实例-编程（3）

◦ ◦ ◦ ◦ ◦ ◦
//3.扫描第3行

◦ ◦ ◦ ◦ ◦ ◦
//4.扫描第4行

PORTD = ~(1 << PORTD4); //PORTD4为0, 其它为1, 送给第4行

_delay_us(2); getkey = (PIND & 0x0f); //取按键阵列的列状态

switch(getkey) { case 0x07:keyno = 12;break; //4行1列编码为12/*
case 0x0b:keyno = 13;break; //4行2列编码为13/0
case 0x0d:keyno = 14;break; //4行3列编码为14/#
case 0x0e:keyno = 15;break; //4行4列编码为15/D }

/*一轮按键阵列处理结束*/

if(keyno < 16) { key_no[3] = key_no[2]; //移动按键数据

key_no[2] = key_no[1];

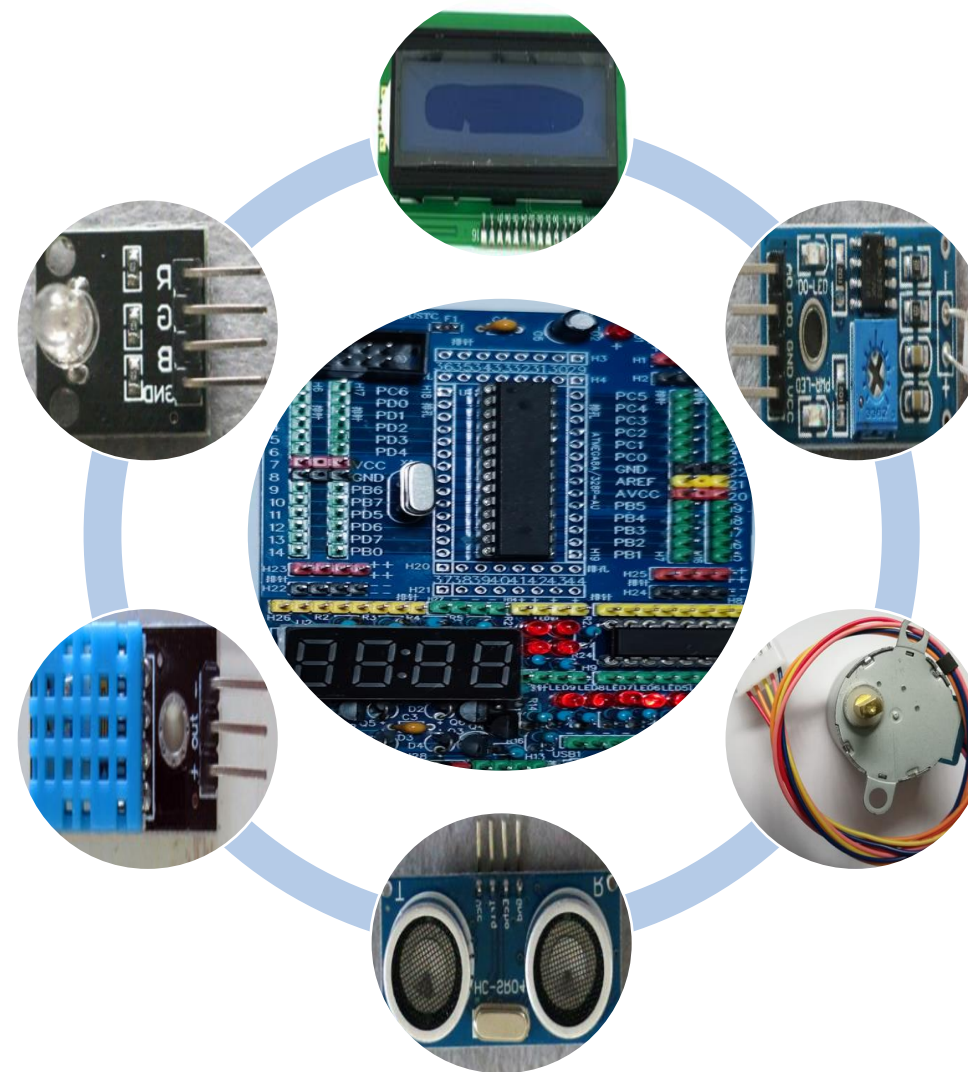
key_no[1] = key_no[0];

key_no[0] = keyno; //新的按键数据 }

 } //ISR结束

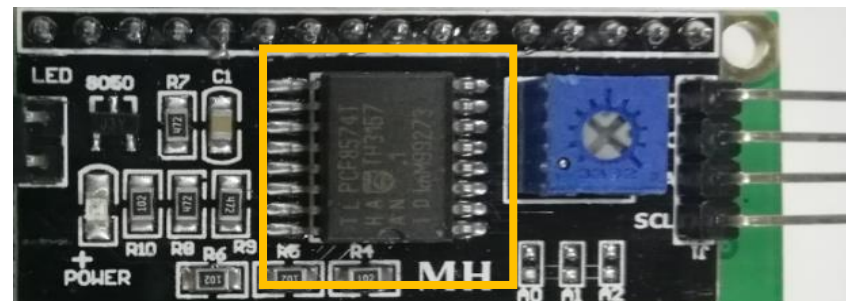
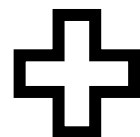
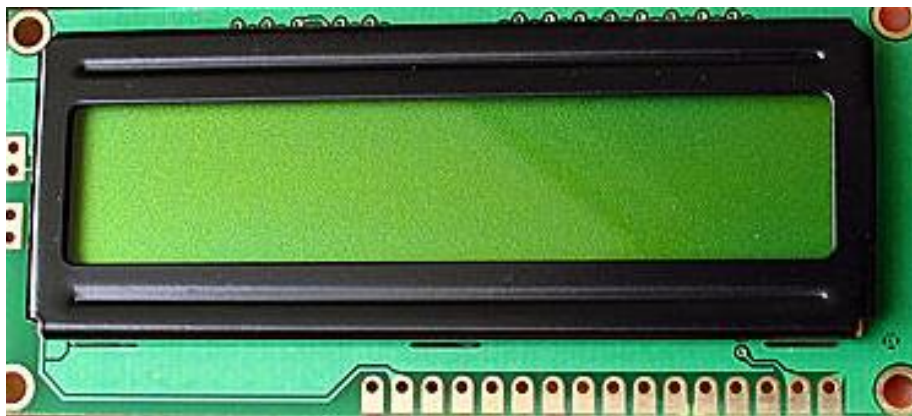


LCD命令与汉字



LCD命令与汉字1：回顾（1）-LCD1602接口

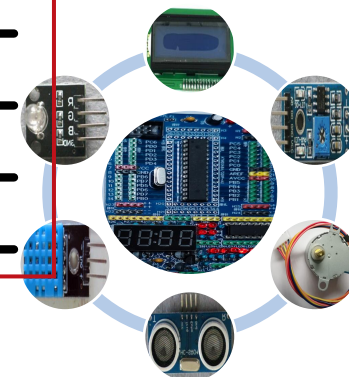
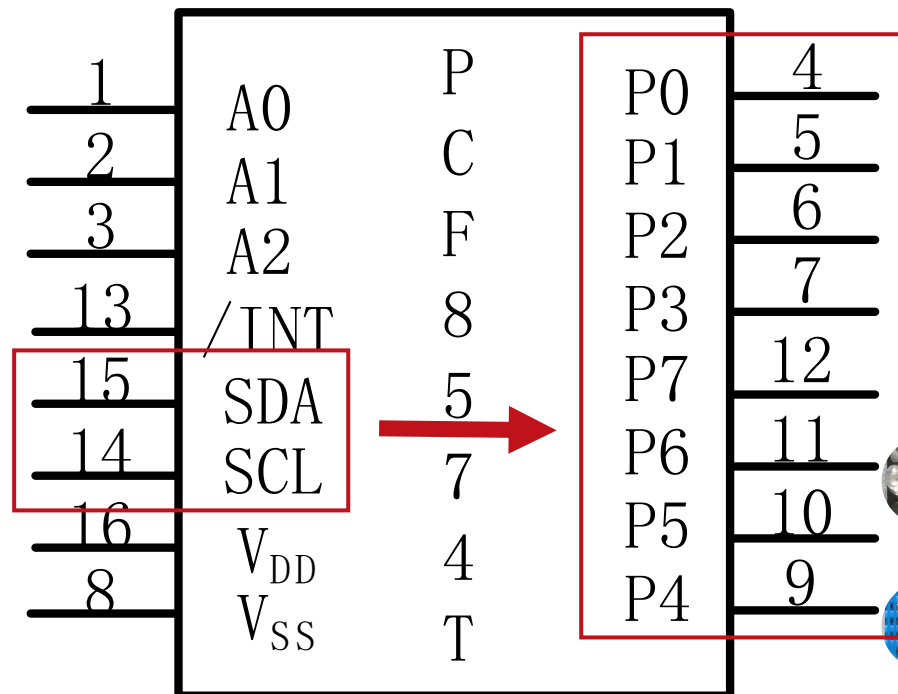
2bits-8bits



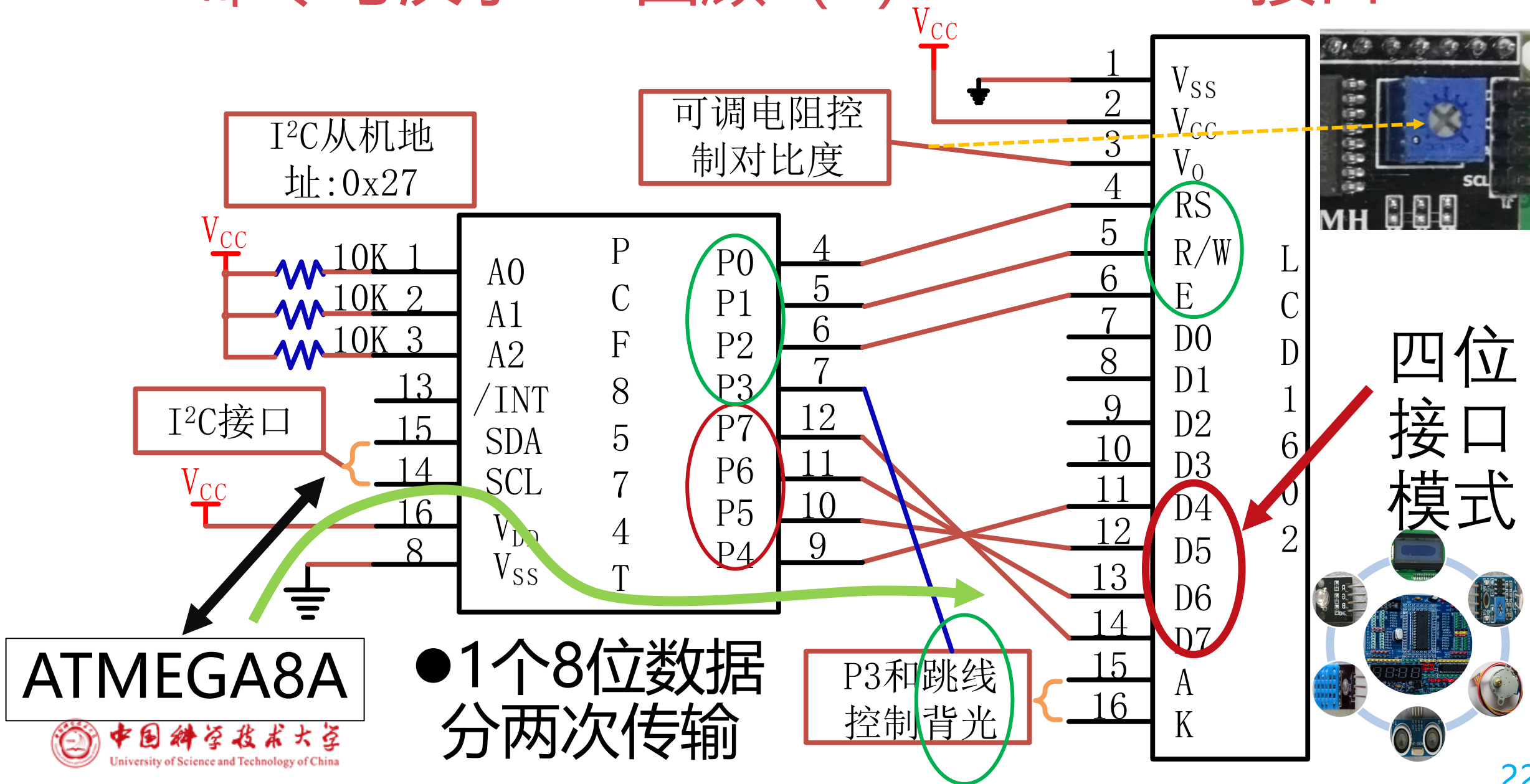
• 需要11/7Pins

支持8位或4位数据传输模式

■ MCU与LCD间仅需2Pins



LCD命令与汉字1：回顾（1）-LCD1602接口2



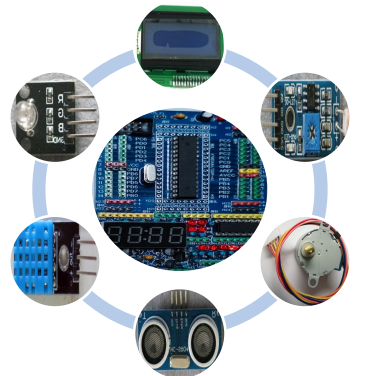
LCD命令与汉字1：回顾（2）-LCD1602显示控制

		1	2	3	4	5	6	...	40
DDRAM 地址	行1	00h	01h	02h	03h	04h	05h	...	27h
	行2	40h	41h	42h	43h	44h	45h	...	67h

■控制器型号为HD44780

■3种存储空间：DDRAM、CGROM和CGRAM

- DDRAM(Display Data RAM) 存储要显示的字符 **ASCII码**
- CGROM(Character Generator ROM)为点阵数据（192）
- CGRAM**存储用户自定义的字符点阵数据(最多8个)

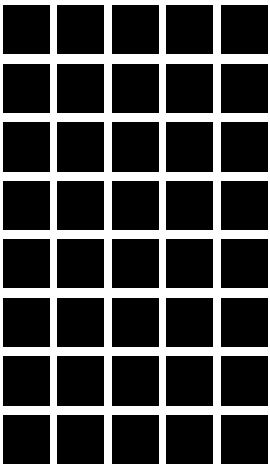


LCD命令与汉字

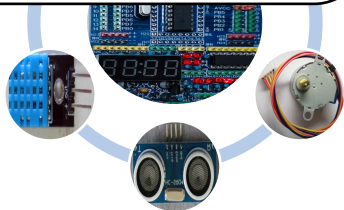
1: 回顾 (3) - 自定义字符与 CGRAM

前8个字符空间地址对应自定义的字符显示

b7- b3 -b0	b4	0000	0010	0011	0100	0101	0110	0111	1010	1011	1100	1101	1110	1111
0000	CG RAM (1)		0	a	P	`	P		-	9	E	α	p	
0001	(2)		!	1	A	Q	a	9	a	ア	チ	4	ä	q
0010	(3)		"	2	B	R	b	r	「	イ	ウ	×	β	θ
0011	(4)		#	3	C	S	c	s	」	ウ	テ	E	ε	∞
0100	(5)		\$	4	D	T	d	t	、	エ	ト	ト	μ	Ω
0101	(6)		%	5	E	U	e	u	・	オ	ナ	ユ	ε	Ü
0110	(7)		&	6	F	V	f	v	ヲ	カ	ニ	ヨ	ρ	Σ
0111	CG RAM (8)		'	7	G	W	g	w	ア	キ	ヌ	ラ	g	π
1000	CG RAM (1)		(	8	H	X	h	x	イ	ク	ネ	リ	フ	Σ
1001	(2)		)	9	I	Y	i	y	ウ	ケ	ル	ル	フ	Y
1010	(3)		*	:	J	Z	j	z	エ	コ	ン	レ	j	千
1011	(4)		+	:	K	[	k	[	オ	サ	ヒ	ロ	*	万
1100	(5)		,	<	L	¥	l	l	ヤ	シ	フ	ワ	Φ	円
1101	(6)		-	=	M	I	m	}	ユ	ズ	へ	ン	ト	÷
1110	(7)		.	>	N	^	n	→	ヨ	セ	ホ	・	円	
1111	CG RAM (8)		/	?	O	_	o	+	ッ	ソ	マ	"	ö	■



CGROM
对应的字
符显示



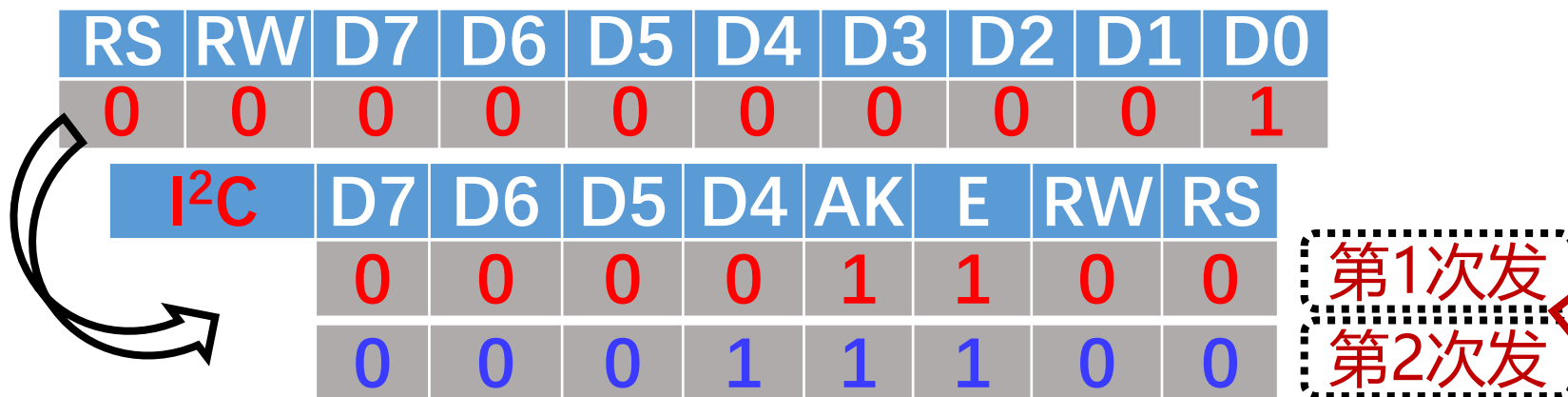
LCD命令与汉字1：回顾（4） -LCD1602指令

指令名称	指令码										说明	时间@ 270KHz
	RS	R/W	D7	D6	D5	D4	D3	D2	D1	D0		
清除屏幕	0	0	0	0	0	0	0	0	0	1	清屏,AC=0,光标复位	1.64ms
光标回原点	0	0	0	0	0	0	0	0	1	x	光标回原点, 显示不变	1.64ms
进入模式设置	0	0	0	0	0	0	0	1	I/D	S	设置光标移动方向等	40us
屏幕开/关控制	0	0	0	0	0	0	1	D	C	B	屏幕,光标等显示切换	40us
光标或显示移位	0	0	0	0	0	1	S/C	R/L	x	x	光标和显示移位控制等	40us
功能设置	0	0	0	0	1	DL	N	F	x	x	8/4位接口,2/1行,5×8/10	40us
置CGRAM地址	0	0	0	1	ACG[5:0]						设置CGRAM地址到AC	40us
置DDRAM地址	0	0	1	ADD[6:0]						设置DDRAM地址到AC	40us	
读忙标志和AD	0	1	BF	AC[6:0]						不论内部是否工作,均可读	40us	
往RAM写数据	1	0	Write Data[7:0]						往CG/DDRAM写数据		40us	
从RAM读数据	1	1	Read Data[7:0]						从CG/DDRAM读数据		40us	
注解	I/D=1:递增模式,I/D=0:递减模式; S=1:移位; S/C=1:显示移位,S/C=0:光标移位; R/L=1:右移,R/L=0:左移; DL=1:8位通信接口,DL=0:4位的; N=1:2行,N=0:1行; F=1:5×10点阵,F=0: 5×8点阵; BF=1:执行内部功能,BF=0:接收命令										AD:Address DDRAM:Display Data RAM CGRAM:Character Generator RAM ACG:CGRAM AD ADD:DDRAM&Cursor AD AC:Address counter for DRAM/CGRAM	
'x'表示不用在意是 '0'还是'1'。												



LCD命令与汉字1：回顾（5） - 4位工作模式

■清除屏幕指令

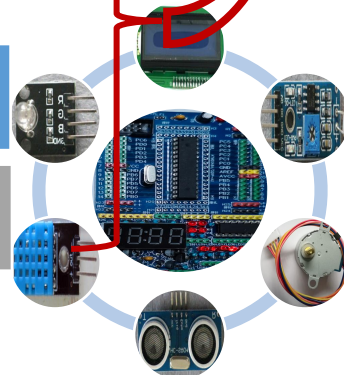


■设置DDRAM地址指令

RS	R/W	D7	D6	D5	D4	D3	D2	D1	D0
0	0	1	A6	A5	A4	A3	A2	A1	A0

■向DDRAM写数据指令

RS	R/W	D7	D6	D5	D4	D3	D2	D1	D0
1	0	8位数据							



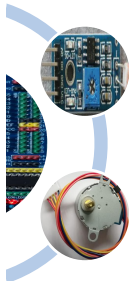
LCD命令与汉字2：字模1-5*8

Character Code (DD RAM Data)								CG RAM Address						Character Patterns (CG RAM Data)									
b7	b6	b5	b4	b3	b2	b1	b0	b5	b4	b3	b2	b1	b0	b7	b6	b5	b4	b3	b2	b1	b0		
0	0	0	0	X	0	0	0	0	0	0	0	0	0	X	X	X	1	1	1	1	1		
											0	0	1				0	0					
											0	1	0				0	0					
											0	1	1				0	0					
											1	0	0				0	0					
											1	0	1				0	0					
											1	1	0				0	0					
											1	1	1				0	0					
0	0	0	0	X	0	0	1	0	0	1	0	0	0	X	X	X	0	1	1	1	0		
											0	0	1				0	0					
											0	1	0				0	0					
											0	1	1				0	0					
											1	0	0				0	0					
											1	0	1				0	0					
											1	1	0				0	0					
											1	1	1				0	0					

Character
Pattern
Example (1)

Cursor
Position

Character
Pattern
Example (2)

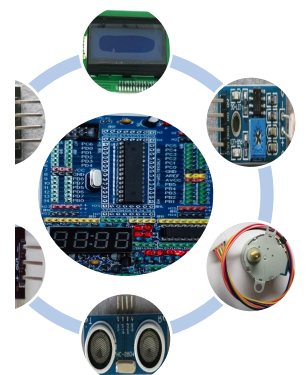


LCD命令与汉字2：字模2-5*10

Character Code (DD RAM Data)								CG RAM Address						Character Patterns (CG RAM Data)							
b7	b6	b5	b4	b3	b2	b1	b0	b5	b4	b3	b2	b1	b0	b7	b6	b5	b4	b3	b2	b1	b0
0	0	0	0	X	0	0	X	0	0	0	0	0	0	X	X	X	1	0	0	0	1
										0	0	0	1				1	0	0	0	1
										0	0	1	0				1	0	0	0	1
										0	0	1	1				1	0	0	0	1
										0	1	0	0				1	0	0	0	1
										0	1	0	1				1	0	0	0	1
										0	1	1	0				1	0	0	0	1
										0	1	1	1				1	0	0	0	1
										1	0	0	0				1	0	0	0	1
										1	0	0	1				1	1	1	1	1
										1	0	1	0				0	0	0	0	0
										1	0	1	1				X	X	X	X	X
										1	1	0	0								
										1	1	0	1								
										1	1	1	0								
										1	1	1	1								

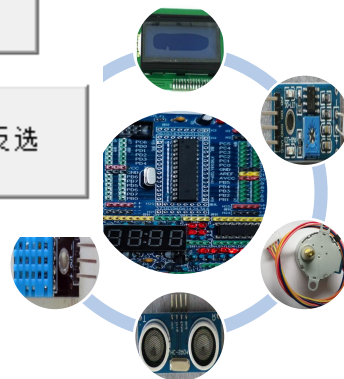
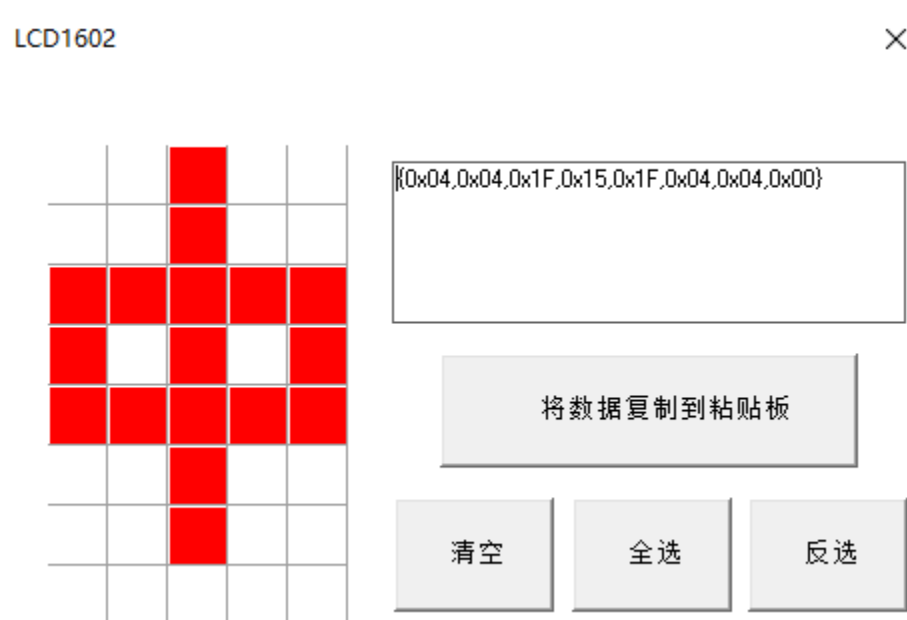
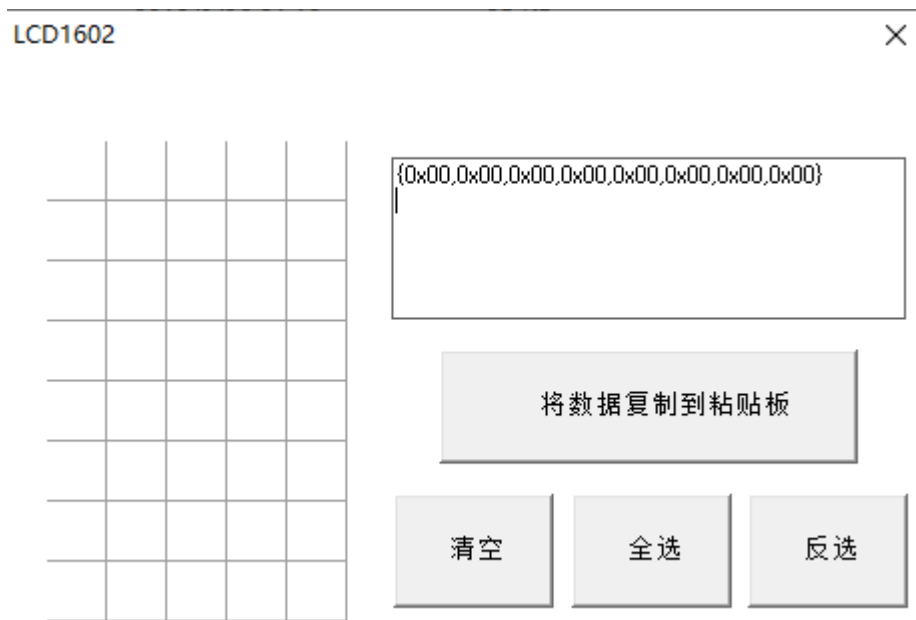
Character
Pattern
Example (1)

Cursor
Position



LCD命令与汉字2：字模3-如何取汉字字模

- 根据5*8或5*10点阵，直接画
- LCD1602小工具



LCD命令与汉字2： 示例-编程（1）

```
unsigned char sf_data[8][8]={           //自定义字符:最多8个, 也可以汉字
{0x04,0x04,0x1F,0x15,0x15,0x1F,0x04,0x04}, //中
{0x1F,0x11,0x1F,0x15,0x1F,0x17,0x1F,0x1F}, //国
{0x0A,0x1E,0x0E,0x1A,0x0F,0x1A,0x0A,0x0A}, //科
{0x00,0x04,0x04,0x1F,0x04,0x00,0x0A,0x11}, //大
{0x00,0x02,0x1F,0x15,0x0A,0x12,0x05,0x09}, //欢
{0x01,0x00,0x07,0x01,0x02,0x02,0x02,0x01}, //迎
{0x04,0x08,0x17,0x15,0x1F,0x15,0x04,0x1F}, //迎
{0x00,0x0A,0x17,0x01,0x12,0x17,0x13,0x16} }; //你

void LCD_CGR_Write()                    //将自定义的8个字符数据写入CGRAM
{ unsigned char i,j;
  for(i=0;i<8;i++) for(j=0;j<8;j++)
  { LCD_8Bit_Write(0x40+i*8+j,0);        //确定地址
    LCD_8Bit_Write(sf_data[i][j],1);     //将对应的自定义字符数据写入
  }
}
```

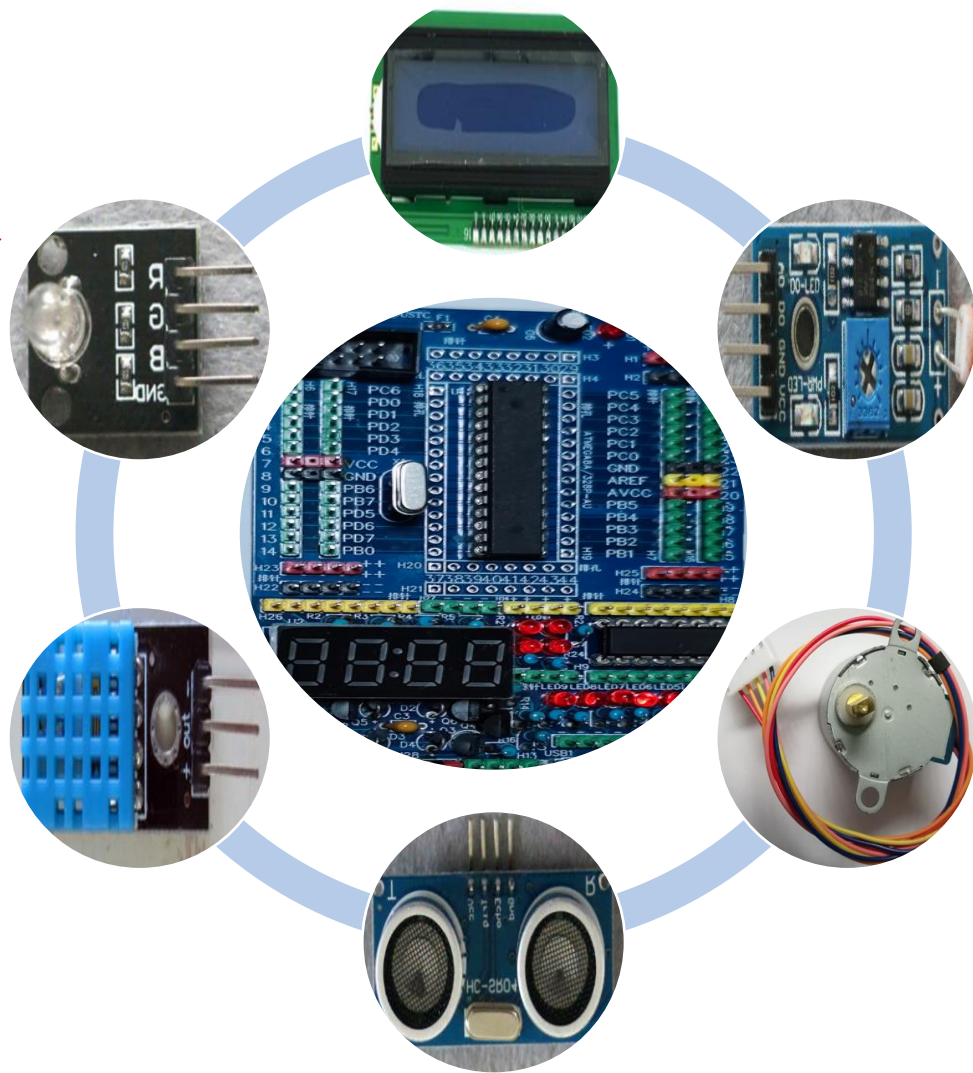


LCD命令与汉字2： 示例-编程 (2)

```
int main(void) { int i;
    TWI_Init(); LCD_Init(); LCD_CGR_Write(); //初始化TWI和LCD, 写入8个自定义
    LCD_Write_String(0,0,"LCD1602 Command 20240430"); _delay_ms(2000);
    LCD_8Bit_Write(0x01,0);/*清屏*/ _delay_ms(2);
    LCD_Write_String(0,0,"LCD1602 Command 20230425");
    LCD_8Bit_Write(0x18,0);_delay_ms(1); //左移一个字符
    LCD_8Bit_Write(0x18,0);_delay_ms(1);
    LCD_8Bit_Write(0x18,0);_delay_ms(1);
    LCD_8Bit_Write(0x18,0);_delay_ms(1);
    LCD_8Bit_Write(0x18,0);_delay_ms(1);
    _delay_ms(2000);
    for(i=0;i<8;i++)LCD_Write_Char(1,i,i); //显示自定义字符
    _delay_ms(2000);
    LCD_8Bit_Write(0x1C,0);_delay_ms(1); //右移一个字符
    LCD_8Bit_Write(0x1C,0);_delay_ms(1);
    LCD_8Bit_Write(0x1C,0);_delay_ms(1);
    LCD_8Bit_Write(0x1C,0);_delay_ms(1);
    LCD_8Bit_Write(0x1C,0);_delay_ms(1);
    while (1) { /*LCD_8Bit_Write(0x18,0);_delay_ms(500); */ }
```



TWI接口的数据传输



TWI接口的数据传输1： 示例-编程（1） MCU1

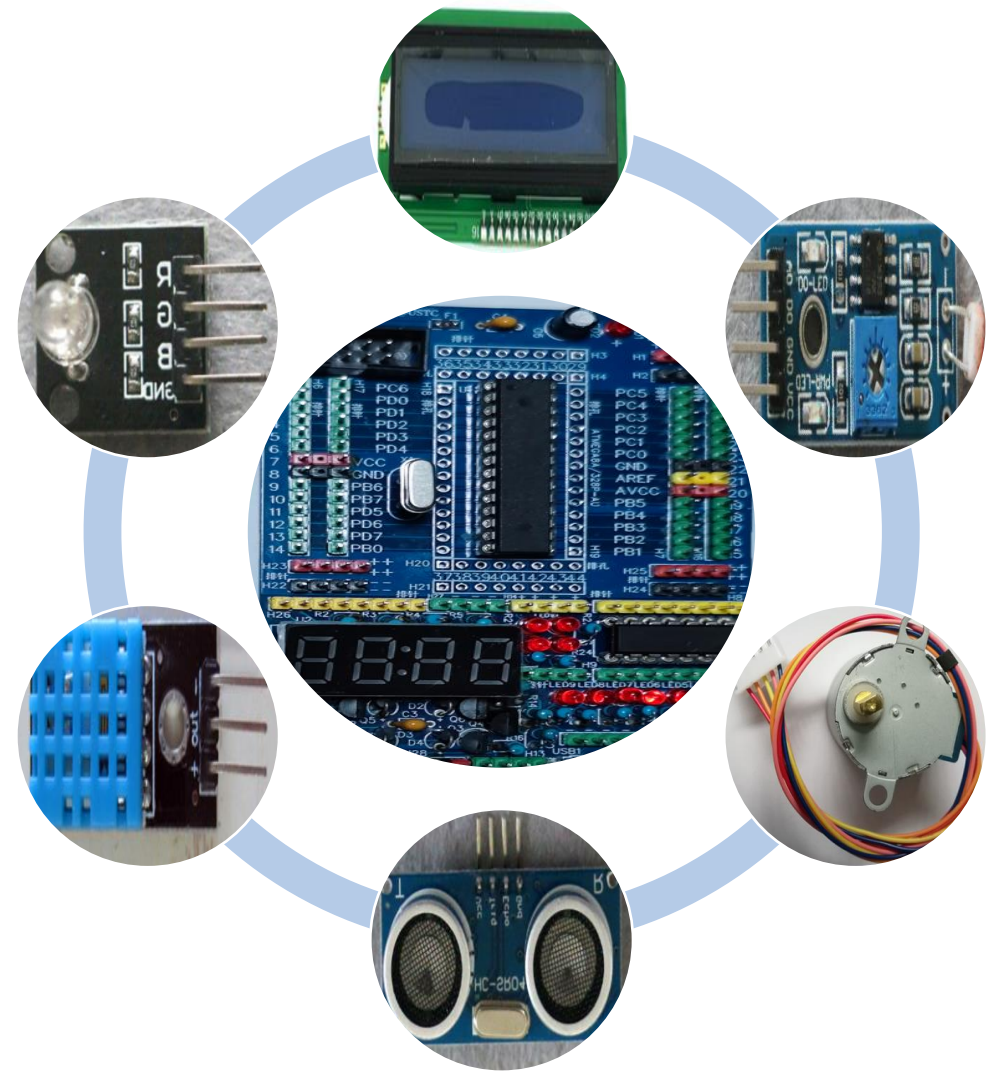
```
DDRB &= ~(1<<DDRB1);           //用PB1接收从设备释放TWI总线没有?
TWI_Init();                      //设置PC4和5, 设置TWI工作速度等

. . .
while (1) {
    while(!(PINB & (1<<PINB1))); //PB1为低电平, 则一直等待, 直到为高电平
    TWI_Start();                 //清除中断标志, 开启TWI接口, 发送START并等待
    if(TWI_Get_State_Info()==TW_START) //START是否正确发出
    {
        TWI_Write(sla_w);        //向总线 (从设备) 的发 (写) 地址, 并等待
        if(TWI_Get_State_Info()==TW_MT_SLA_ACK) //正确写/发了从设备地址?
        {
            TWI_Write(counter);    //向总线 (从设备) 的发 (写) 数据, 并等待
            if(TWI_Get_State_Info()==TW_MT_DATA_ACK) //数据已发出
                TWI_Stop();        //发送STOP, 最后关闭TWI
        }
    }
    TWCR=(1<<TWINT);             //不管如何都关闭TWI
}
```


TWI接口的数据传输1：示例-编程（2）MCU2

```
DDRB = (1<<DDRB0);           //用PB0告诉主设备，从设备释放TWI总线没有？
PORTB &= ~(1<<PORTB0);        //告诉主设备，从设备将要占用TWI总线
TWI_Init(); LCD_Init();      。 。 。
while (1) {
    PORTB |= (1<<PORTB0);      //告诉主设备，从设备已经释放TWI总线
    TWI_Slv_Ready();           //开启从设备的TWI，等待并应答主设备寻址本设备
    if(TWI_Get_State_Info()==TW_SR_SLA_ACK) //s1a+W已收到，已发ACK
    {
        TWI_Slv_Ready();      //从设备再次准备好
        if(TWI_Get_State_Info()==TW_SR_DATA_ACK) //正确接收数据并发ACK？
        {
            counter = TWDR;
            PORTB &= ~(1<<PORTB0); } //告诉主设备，从设备将要占用TWI总线
    }
    TWCR = (1<<TWINT);        //不管如何，清除标志，关闭TWI接口
    。 。 。
    LCD_Write_String(0,2,"Touchpad Times:");LCD_Write_String(1,2,chs);
}
```

蜂鸣器发声

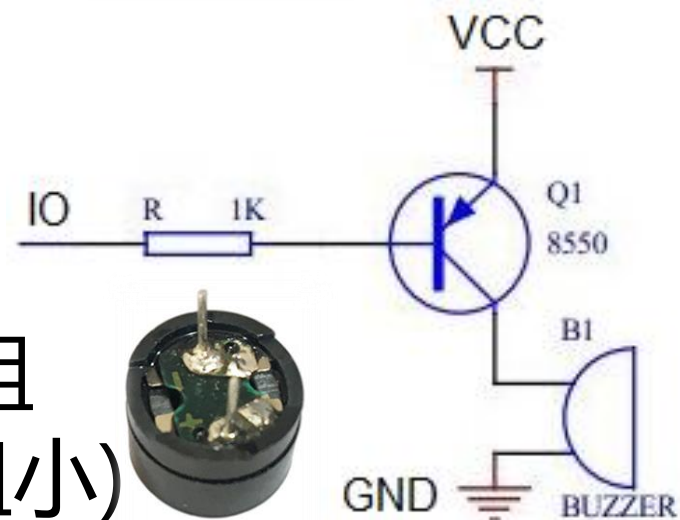
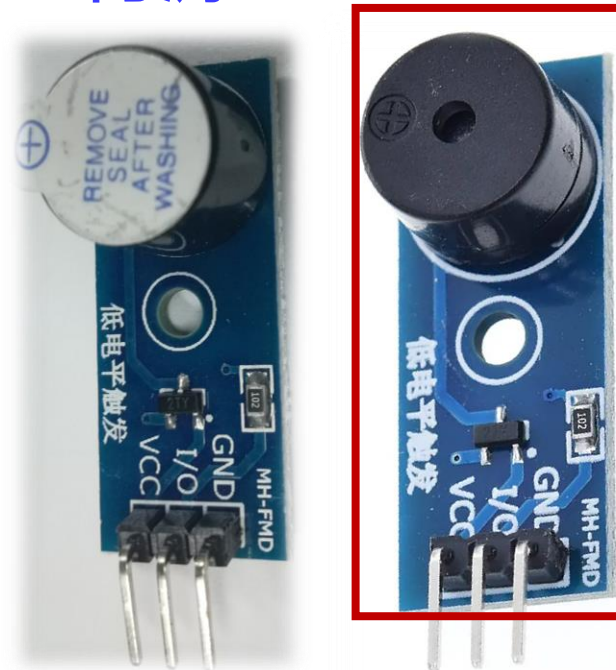


蜂鸣器发声1：蜂鸣器概述

- 有源蜂鸣器：
 - 内部带震荡源，只要一通电就会响
 - 程序控制方便，单片机IO管脚输出一个低电平到蜂鸣器就可以让其发出响声
- 无源蜂鸣器：
 - 内部不带震荡源，用**直流信号**无法令其鸣叫。必须用一定频率的方波信号去驱动它
 - 声音**频率可控**，可以做出“多来米发索拉西”的效果
- 如何判断：加电（有源持续响），测量电阻（无源电阻小）

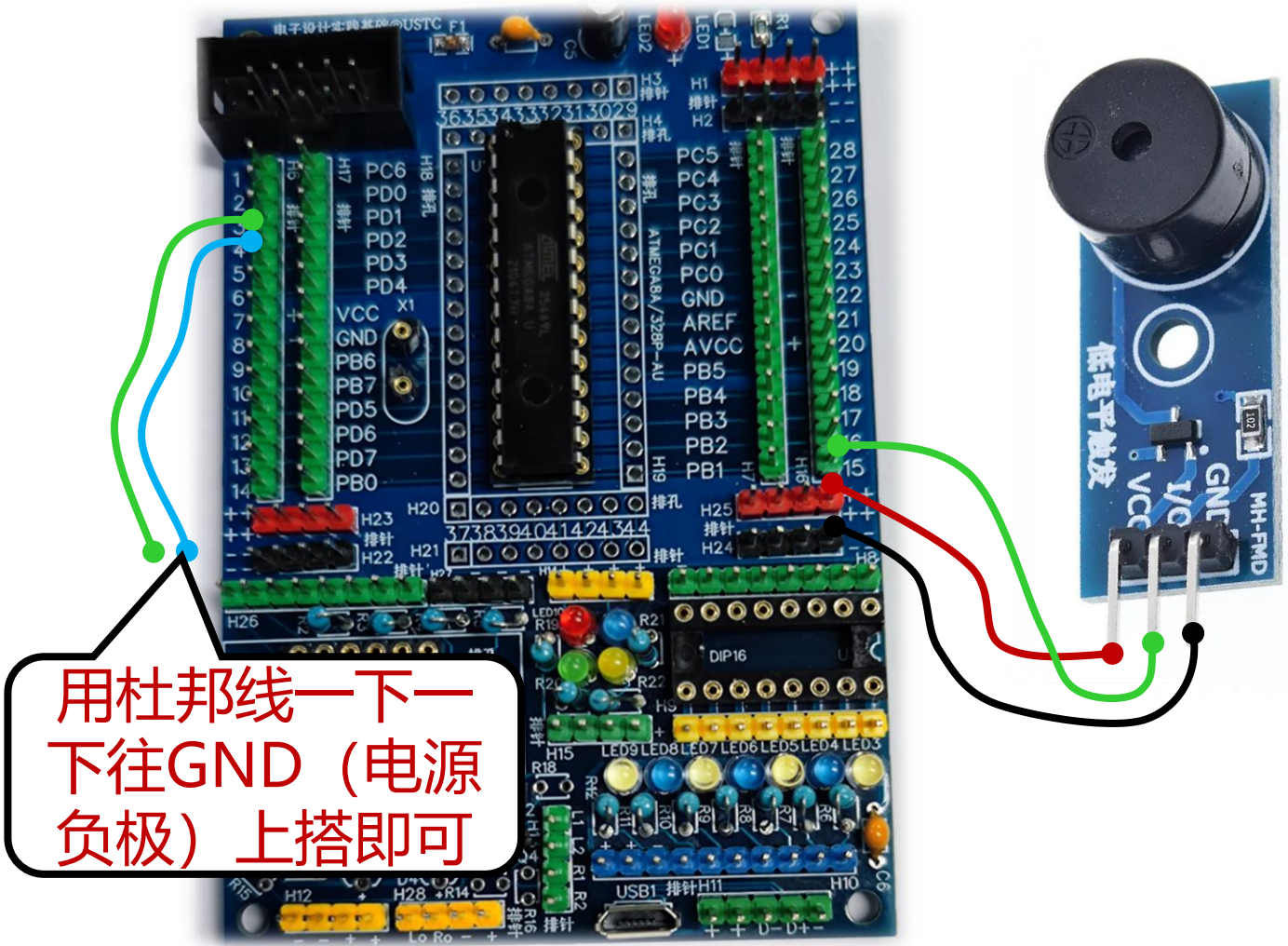
有源

无源



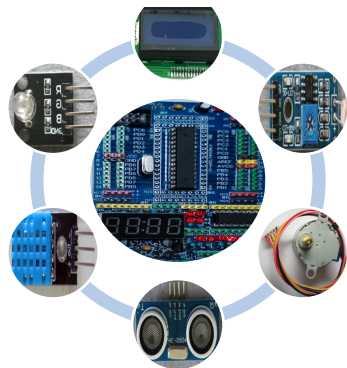
蜂鸣器发声2：实例-硬件连接

C调音符与频率对照表



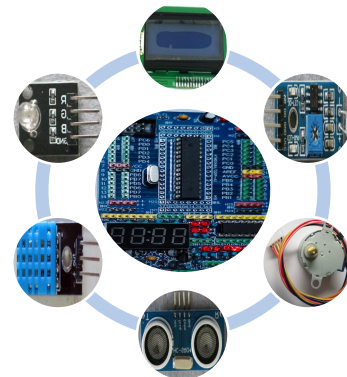
音符	频率/Hz	音符	频率/Hz	音符	频率/Hz
低音1	262	中音1	523	高音1	1046
低音1#	277	中音1#	554	高音1#	1109
低音2	294	中音2	587	高音2	1175
低音2#	311	中音2#	622	高音2#	1245
低音3	330	中音3	659	高音3	1318
低音4	349	中音4	698	高音4	1397
低音4#	370	中音4#	740	高音4#	1480
低音5	392	中音5	784	高音5	1568
低音5#	415	中音5#	831	高音5#	1661
低音6	440	中音6	880	高音6	1760
低音6#	466	中音6#	932	高音6#	1865
低音7	494	中音7	988	高音7	1976

CSDN @Menida



蜂鸣器发声2：实例-编程（1）

```
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
unsigned char duty=50; //占空比, 百分比
unsigned int freq=30; //频率: 30~3000Hz范围可调( 实测29.8~2.7K)
ISR(INT0_vect)
{ if(duty > 90) //百分比
    duty = 10;
  else duty +=10; }
ISR(INT1_vect)
{ if(freq >2000) freq = 30;
  else freq +=100; }
```



蜂鸣器发声2：实例-编程（2）

```
int main(void) {  
    DDRD &= ~((1 << DDRD2) | (1 << DDRD3)); //分别调整占空比和频率  
    PORTD |= (1 << PORTD2) | (1 << PORTD3); //开启内部上拉电阻，默认高电平  
    DDRB |= (1 << DDRB1); //PB1控制蜂鸣器的IO  
    MCUCR |= (1 << ISC01) | (1 << ISC11); //INT0和INT1下降沿触发  
    GICR |= (1 << INT0) | (1 << INT1); //开中断  
    sei(); //全局中断开  
    unsigned int high, low, i;  
    while (1) {  
        high = F_CPU/14/freq/100*duty; //换算为高电平的周期数  
        low = F_CPU/14/freq - high - 160; //换算为低电平的周期数  
        PORTB |= (1 << PORTB1); for(i=0; i<high; i++) _delay_us(1); //高电平持续  
        PORTB &= ~(1 << PORTB1); for(i=0; i<low; i++) _delay_us(1); //低电平持续  
    }  
}
```



蜂鸣器发声2：实例2-编程（1）

```
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
unsigned int t_icr1=0;
//记录TOP
ISR(INT1_vect) {
    if(ICR1 < 200)
//200~25000:2.5K~20Hz（实测
19.8~2.5K）
        ICR1 = 25000;
    else { ICR1 -=t_icr1/20;
    }
    t_icr1 =ICR1;
    OCR1A = ICR1/2;
}
```

```
int main(void) {
    DDRB |= (1<<DDRB1);           //PB1(OC1A)输出
    ICR1 = 25000;                  /*TOP值*/
    t_icr1 = ICR1;
    OCR1A = ICR1/2;               //A路比较值,默认50%
    TCCR1A |= (1<<COM1A1)|(1<<COM1A0); //管脚切换方式
    TCCR1B |= (1<<WGM13)|(1<<CS11);    //模式为8, 时钟8分频
    DDRD &= ~(1<<DDRD3);           //PD3 tp223控制PWM频率
    MCUCR |= ((1<<ISC11)|(1<<ISC10)); //int1上升沿触发中断
    GICR |= (1<<INT1);             /*允许INT1中断*/
    sei();                        //开全局中断
    while (1) {
        _delay_ms(2);
    }
}
```



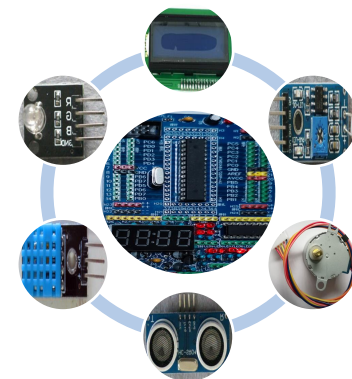
本周实验内容

实验内容1：通过简单的任务调度，完成3个及以上的任务调度

实验内容2：在LCD1602上显示本人的学号与中文名字

实验内容3：在无源蜂鸣器上模拟演奏一段乐谱

自行练习

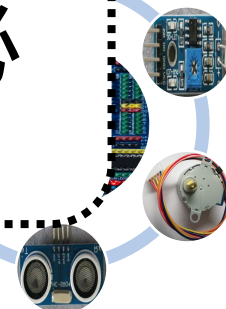


综合实验

1, **11, 12周**正常的实验时间可到实验室做实验, 问问题, 验收等; 最迟**5月17日晚上10点前**到实验室验收、**5月19日晚上24点前**提交**设计报告**等(报告、代码)

2, **综合设计**要求: 基于课程内容, 独立完成一个电子系统的设计, 要完整, 可以运行; 非必要不用课程以外的模块 (不含课堂等额外发放的)

3, **设计报告**要求: 设计思路、过程, 必要的流程与代码分析, 结果与分析, 拓展等。代码要有必要的注释。最后把报告和代码等一起打包提交



谢 谢

